Loopverse 3.0 · Hackathon Question Paper

ExamSlot: A Self-Service Exam Date Sheet System for a Multi-Branch Virtual University

Build a full-stack, responsive web application where administrators manage branches, courses, students and exam schedules, and students design their own date sheet, exactly once.

Total marks	Team size	Duration	Platforms
100 (+10 bonus)	**__ to __ members**	**__ hours**	**Web and mobile browsers**

1. Background

A Virtual University has several campuses (branches) across the country. Its students sit exams in person, but instead of the university publishing a fixed date sheet, each student designs their own: they choose which branch they will sit their papers in, and for each enrolled course they pick one of the exam dates and times that the administration has opened for it.

Your task is to build this system end to end. The hackathon expects a working, demo-ready product, not a set of mock screens.

2. Problem statement

Design and develop **ExamSlot**, a web application with two panels, an **Admin Panel** and a **Student Panel**, that together support the complete life cycle of a self-designed exam date sheet: setup by the admin, registration and onboarding of students, course assignment, schedule publishing, student selection, printing, and controlled change requests.

3. Roles

Role	What they can do
Admin	Manages branches, courses, students, course assignments and exam schedule slots. Reviews student requests and approves or rejects them.
Student	Logs in with email and password, selects an exam branch once, chooses a date and time for each assigned course, saves and prints the date sheet, and raises change requests.

4. Admin Panel requirements

4.1 Branch management (full CRUD)

- Create, view, update and delete branches (the campuses where exams can be taken). Suggested fields: name, code, city, address, contact number, status (active or inactive).
- A student may sit their exams in *any* active branch.

- A branch that is already chosen by students cannot be hard-deleted. Use a safe-delete rule (block the delete or mark it inactive) and explain your choice in the README.

4.2 Course management (full CRUD)

- Create, view, update and delete courses. Suggested fields: course code (unique), title, credit hours, department, status.

4.3 Student management (full CRUD)

Each student record must capture complete information in three groups:

Group	Minimum fields
Personal	Full name, email (unique), phone, CNIC or B-Form number, date of birth, gender, address, profile photo (optional)
Parent / guardian	Father or guardian name, parent CNIC, occupation, contact number, emergency contact
Academic	Registration number (unique), program or degree, semester, session or batch, previous qualification, previous institute, marks or CGPA

> **Account creation email.** The moment a student is created, the system sends an email to the address on file. The email explains that a portal account has been created and contains a secure link to set a password. The student opens the email, resets (sets) their password, and from then on logs in with their email and password.
>
> The link must be single-use and time-limited (for example 24 hours). Passwords must never be stored or emailed in plain text.

4.4 Course assignment (full CRUD)

- The admin assigns courses to each student, **at least 4 and at most 6.**
- The rule is enforced on the server, not only in the interface. A student with fewer than 4 courses is shown as "assignment incomplete" and cannot proceed to date sheet selection.
- The same course cannot be assigned to the same student twice.
- The admin can view, change and remove assignments, as long as the 4 to 6 rule stays valid and the student has not yet saved a date sheet. Decide and document what happens after a date sheet is saved (the recommended rule is that changes are blocked unless the admin approves a date sheet change request).

4.5 Exam schedule management (full CRUD)

- For every course, the admin creates one or more exam slots, each with a **date** and a **time** (start time, and optionally an end time).
- The student chooses from these slots, so offering more than one slot per course gives the student real choice.
- The admin can view, edit and delete slots. Slots that have already been chosen by a student must be protected (block the delete, or warn and notify the affected students).
- Validation: no slot in the past, no duplicate slot for the same course, end time after start time.

4.6 Student requests

- A list of all requests raised by students, with student name, request type

(*change branch* or *change date sheet*), reason, date raised and status (pending, approved, rejected).

- The admin can approve or reject each request, with an optional remark.
- Filter by status and type.

4.7 Search and pagination

Every listing in the Admin Panel (branches, courses, students, assignments, schedules, requests) must have **search and pagination**. Pagination must be done on the server side (page and page size are sent to the API and only that page of data is returned). Search results must also be paginated and the page counter must reflect the filtered total. Loading all records and slicing them in the browser will lose marks.

---

5. Student Panel requirements

5.1 Login

Students log in with their email and password. Students cannot register themselves; accounts exist only when the admin creates them. Provide a "forgot password" flow that uses the same email reset mechanism.

5.2 Branch selection (one time only)

- After the first successful login, the student sees a page to select the branch where they want to take their papers.
- The selection can be made **only once**. Once saved, this page is never shown again on later logins, and the student goes straight to the next page.
- The rule must be enforced by the backend too. Calling the API a second time

must be rejected.

5.3 Dashboard and date sheet design page

- The student sees their complete information (personal, parent and academic) in read-only form, along with the selected branch.
- Below it, all courses assigned to the student are listed.
- For every course the student selects one **date** and one **time** from the slots the admin created for that course.
- The student must select a slot for every assigned course before saving.
- Conflict rule: the student cannot pick two courses at the same date and overlapping time. Show a clear message and block saving.

5.4 Save, view and print

- After the student saves, a **proper date sheet** is shown, containing the student name, registration number, program, branch and a table of course code, course title, date, day and time, sorted by date.
- A **Print Date Sheet** button appears and produces a clean printable page (a print stylesheet or PDF export is fine).
- After saving, the date sheet **cannot be created or changed again**, and the branch cannot be changed again, unless an admin approves a request (see below).

5.5 Need help: change requests

The student has a **Need Help** option with two choices:

1. Request to change branch
2. Request to change date sheet

- The student writes a reason and submits. The request appears in the Admin Panel as pending.
- The student can see the status of their requests (pending, approved, rejected) and the admin's remark.
- If the admin **approves**, the next time the student logs in the relevant action is unlocked: a branch change reopens the branch selection page, and a date sheet change reopens the date sheet page.
- The unlock is **one time**. After the student uses it, it locks again.
- If the admin **rejects**, nothing changes and the student sees the rejection with the remark.
- A student cannot have two pending requests of the same type at once.

---

6. Student journey at a glance

1. Admin creates the student. A password setup email is sent automatically.
2. Student opens the email, sets a password, and logs in.
3. Student selects an exam branch. This happens one time only.
4. Student sees their profile and assigned courses, then picks a date and time for each course.
5. Student saves. The date sheet is generated and can be printed. Selection is now locked.
6. If something must change, the student raises a request. After the admin approves it, the matching action unlocks once.

7. Business rules (quick reference)

Rule	Enforced where
4 to 6 courses per student	Backend and UI
Branch can be selected once; the selection page does not reappear	Backend and UI
Date sheet can be saved once	Backend and UI
Student picks only from slots the admin created for that course	Backend
No two courses of a student in the same date and time	Backend and UI
Unlock after admin approval is single use	Backend
Students see only their own data; admin routes are closed to students	Backend (role based access)
Unique email, registration number, branch code and course code	Database and validation

8. Non-functional requirements

- **Responsive:** fully usable on phones, tablets and desktops. Tables must scroll or collapse into cards on small screens. Test at 360 px width.
- **Security:** hashed passwords, token or session based authentication, role based authorization on every route, input validation, and protection against common attacks (SQL injection, XSS)

common attacks (SQL injection, XSS).

- **Usability:** loading states, clear error messages, confirmation before deleting, and success feedback after actions.
- **Data integrity:** use proper relations and constraints (students, courses, branches, assignments, slots, selections, requests).
- **Code quality:** organized folder structure, meaningful names, environment variables for secrets, and no hard-coded credentials.

---

9. Technology

Teams are free to choose their stack (for example MERN, Laravel, Django, Spring Boot, .NET, or Next.js with any SQL or NoSQL database). Use any email service for the password setup mail (SMTP, Gmail app password, Resend, SendGrid, Mailtrap and so on). A hosted demo is preferred, but a local run with clear instructions is accepted.

---

10. Deliverables

- Source code in a public or shared GitHub repository, with regular commits from all members.
- A `README` with setup steps, environment variables, a database diagram (ERD), and the assumptions your team made.
- Seed data or a seed script containing at least: 1 admin, 3 branches, 8 courses, 5 students and a few slots.
- Admin demo credentials and one student demo credential in the README.
- A live link (if hosted) and a demo of 5 minutes.

11. Evaluation

Criteria	Marks
Admin: branch, course and student CRUD with full student information	15
Email based password setup and login	10
Course assignment with the 4 to 6 rule	8
Exam schedule (date and time) CRUD with validation	10
Search and server side pagination	7
Student: one time branch selection and page flow	8
Student: course wise date and time selection, conflict check, locking after save	10
Date sheet view and print	5
Need Help requests and admin approve or reject, with one time unlock	10
Responsive design (web and mobile)	7
Security, validation and code quality	5
Presentation, documentation and demo	5
Total	**100**

Bonus (up to 10 marks)

- Branch seat capacity per slot, with the slot disappearing when full (3)
- Downloadable PDF date sheet (2)
- Email notification to the student when a request is approved or rejected (2)
- Admin dashboard with counts and charts (for example students who have saved their date sheet, pending requests) (2)
- Dark mode or an audit log of admin actions (1)

12. Rules for participants

- All work must be done during the hackathon window. Pre-built projects will be disqualified.
- Libraries, templates and AI assistance are allowed, but the team must be able to explain every part of the code during the demo.
- Where this paper is silent, make a sensible assumption and write it in the README. Judges reward clear reasoning.
- Late submissions will not be evaluated.

Loopverse 3.0 · Question paper. Fill in the team size and duration before distributing.